

CHAPTER 1: INTRODUCTION TO PLATFORM TECHNOLOGIES

Platform Technologies

- Foundational systems that support software applications
- Enables software functionality, interaction, and communication
- Operating System, Cloud, Mobile, Web

Virtualization

- It allows multiple virtual machines (VMs) to run on a single physical machine by abstracting the hardware resources.

Hypervisor

- A software that creates and runs the virtualization.

TYPE of Hypervisor:

TYPE I - no existing software or OS installed in the machine

TYPE II - installed and run on top of an existing OS

Cloud Computing Platform

- It provides on-demand computing resources over the internet. These resources can include anything from storage and networking to databases and machine learning models.
- Cloud platforms are transforming how businesses operate by providing scalable, cost-effective solutions that can be accessed from anywhere in the world. They support everything from simple website hosting to complex machine learning models.

SERVICE MODELS:

- **Infrastructure as a Service (IaaS):** Provides virtualized computing resources over the internet. Users can rent virtual machines, storage, and networks without managing the underlying hardware.  
The cloud provider will manage the servers, storage, virtualization, and networking portion, while the user still has control over the software portion (app, data, OS, middleware, runtime)
- **Platform as a Service (PaaS):** Offers a platform that allows developers to build, run, and manage applications without worrying about the underlying infrastructure.  
It allows the cloud provider to manage a portion of your business, but it has more control  
It also manages the OS, middleware, and runtime, while users are responsible for the app and data
- **Software as a Service (SaaS):** Delivers software applications over the internet, eliminating the need for local installation and maintenance.  
All apps are hosted by the cloud provider.  
The user simply accesses and runs the app on the computer when the cloud service is connected through the internet.  
The cloud provider manages all the hardware, software, networking, OS, and storage.

|      | Description                               | Cost   | Example           |
|------|-------------------------------------------|--------|-------------------|
| IaaS | Provides virtualized hardware             | High   | AWS EC2           |
| PaaS | Provides runtime and tools for developers | Medium | Google App Engine |
| SaaS | Fully managed apps over the internet      | Low    | Gmail, Canva      |

API (Application Programming Interface)

- A set of rules and tools that allows one piece of software to interact with another. It defines the methods and data structures that developers can use to communicate with a service or platform.
- It enables the integration of different systems, allowing them to work together seamlessly. For instance, a mobile app can use an API to access a remote server’s data, or a web application can use an API to retrieve information from a database.

ARCHITECTURE OF PLATFORM TECHNOLOGIES

Layered Architecture

- This architecture divides the platform into layers, where each layer has a specific role and interacts with the layers directly above and below it.

## IT 412 Platform Technologies

- This ensures modularity, reusability, and separation of concerns, making systems easier to manage, develop, and troubleshoot.
  - **Presentation Layer** (e.g., front-end web pages) interacts with users.
  - **Application Layer** (e.g., business logic written in Python or Java) processes user requests.
  - **Data Layer** (e.g., SQL or NoSQL databases) manages persistent data storage.

### Client-Server Model

The client-server model is fundamental in networked environments where a central server provides resources or services, and multiple clients consume them.

### Microservices Architecture

An application is broken down into small, loosely coupled, independently deployable services.

Example: **Netflix** uses a microservices architecture to manage its vast library of content, ensuring high availability and scalability by deploying services across multiple AWS regions.

### Monolithic

It is built as single units where all components (UI, business logic, database access) are tightly coupled. While easier to develop initially, monolithic architectures face challenges in scalability, flexibility, and deployment.

### Key Components of Platform Technologies

- **Operating Systems** – manage hardware resources (CPU, memory, disk, network) and provide essential services like process management, security, and file management.  
*Linux* (used extensively in servers and cloud environments)  
*Windows* (dominant in enterprise desktops)  
*macOS* (popular in creative and development environments)
- **Middleware** – a software that provides common services and capabilities to applications outside of what's offered by the operating system. It acts as a bridge between applications and the OS/network, handling tasks like communication, data management, messaging, and authentication.
- **Databases** – crucial for managing structured, semi-structured, and unstructured data within platforms. They provide capabilities for data storage, retrieval, and management.
- **Virtualization and Cloud Environments** – enables multiple VMs to run on a single physical machine, improving resource utilization. Cloud environments provide on-demand resources over the internet with scalable, flexible, and cost-effective solutions.

**APIs (Application Programming Interfaces)** provide a standardized way for different software components to interact.

**Protocols** like HTTP/HTTPS, TCP/IP, and WebSocket are essential for communication over networks, ensuring reliable and secure data exchange.

## CHAPTER 2: OPERATING SYSTEM AND APPLICATION

### Operating System

- Acts as an **intermediary** between the user of a computer and the computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs in a convenient and efficient manner.
- Bridge between the hardware (CPU, memory, storage) and applications
- The **manager** of the entire device

What does the OS do?

- **USER VIEW**
  - OS is the interface that makes the computer easy to use.
  - It lets users give commands through a graphical interface (clicks, menus, icons) or a command line.

- **SYSTEM VIEW**

- OS is the resource manager that controls and coordinates hardware.
- It manages the CPU, memory, storage, and input/output devices.
- It ensures multiple programs can run efficiently and securely without interfering with each other.

## COMPUTER SYSTEM ARCHITECTURE

### 1. Single Processor Systems

- One main CPU executes instructions.
- All tasks are handled by this single processor.

### 2. Multiprocessor Systems (Parallel Systems)

- Two or more CPUs that share the same memory and work together.
- Tasks are divided among processors to achieve higher performance and reliability.
  - **Symmetric Multiprocessing (SMP):** All processors share the same OS and work equally.
  - **Asymmetric Multiprocessing (AMP):** One processor controls the system, others perform assigned tasks.

## OS OPERATION:

### Multiprogramming & Multitasking

#### **Multiprogramming:**

- OS keeps multiple programs in memory at the same time.
- The CPU switches between them, so when one program waits (e.g., for input/output), another can run.
- The goal is to maximize CPU utilization (no idle time).
- Example: While a file is downloading, the OS lets you type in a text editor.

#### **Multitasking:**

- OS allows multiple tasks (programs) to run seemingly at the same time.
- It rapidly switches the CPU among tasks, giving the user the feeling of parallel execution.
- The goal is to improve user experience by making the system responsive.
- Example: Listening to music while browsing the internet and editing a document.

### Types of Operating Systems

#### 1. Batch Operating System

- ✓ The users of a batch operating system do not interact with the computer directly. Each user prepares his job on an off-line device like punch cards and submits it to the computer operator.
- ✓ To speed up processing, jobs with similar needs are batched together and run as a group. The programmers leave their programs with the operator and the operator then sorts the programs with similar requirements into batches.

#### 2. Time-sharing Operating Systems

- ✓ Time-sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time.
- ✓ Timesharing or multitasking is a logical extension of multiprogramming.
- ✓ Processor's time which is shared among multiple users simultaneously is termed as timesharing.
- ✓ The objective is to minimize response time.
- ✓ The operating system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time. Computer systems that were designed primarily as batch systems have been modified to time-sharing systems.
- ✓ Advantages of Timesharing operating systems are as follows
  - Provides the advantage of quick response.
  - Avoids duplication of software.
  - Reduces CPU idle time

3. Distributed Operating System

- ✓ Distributed systems use multiple central processors to serve multiple real-time applications and multiple users.
- ✓ The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines). These are referred as loosely coupled systems or distributed systems. Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, computers, and so on.
- ✓ The advantages of distributed systems are as follows
  - With resource sharing facility, a user at one site may be able to use the resources available at another.
  - Speedup the exchange of data with one another via electronic mail. • If one site fails in a distributed system, the remaining sites can potentially continue operating.
  - Better service to the customers.
  - Reduction of the load on the host computer.
  - Reduction of delays in data processing.

4. Network Operating System

- ✓ A Network Operating System runs on a server and provides the server the capability to manage data, users, groups, security, applications, and other networking functions.
- ✓ The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.
- ✓ The advantages of network operating systems are as follows
  - Centralized servers are highly stable.
  - Security is server managed.
  - Upgrades to new technologies and hardware can be easily integrated into the system.
  - Remote access to servers is possible from different locations and types of systems.

5. Real-Time Operating System

- ✓ A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment.
- ✓ The time taken by the system to respond to an input and display of required updated information is termed as the response time. So in this method, the response time is very less as compared to online processing.
- ✓ Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application.
- ✓ A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail.
- ✓ For example, Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Summary:

| Types of OS            | Description                                                                     |
|------------------------|---------------------------------------------------------------------------------|
| <b>Batch OS</b>        | Executes batches of jobs efficiently; not suitable for time-critical operations |
| <b>Time-Sharing OS</b> | Allows multiple users to interact simultaneously.                               |
| <b>Distributed OS</b>  | Coordinates multiple computers to work as one system.                           |
| <b>Network OS</b>      | Manages shared network resources like printers and files                        |
| <b>Real-Time OS</b>    | Responds to inputs within a guaranteed time.                                    |

Process Management

In multiprogramming environment, the OS decides which process gets the processor when and for how much time. This function is called Process Scheduling. An Operating System does the following activities for processor management:

- ✓ Keeps tracks of processor and status of process. The program responsible for this task is known as traffic controller.
- ✓ Allocates the processor (CPU) to a process.
- ✓ De-allocates processor when a process is no longer required.
- ✓ Ensuring that each process and application receives enough of the processor’s time to function properly
- ✓ Using as many processor cycles as possible for real work

### Memory Management

Memory management refers to management of Primary Memory or Main Memory. An Operating System does the following activities for memory management:

- ✓ Keeps tracks of primary memory, i.e., what part of it are in use by whom, what part are not in use.
- ✓ In multiprogramming, the OS decides which process will get memory when and how much.
- ✓ Allocates the memory when a process requests it to do so.
- ✓ De-allocates the memory when a process no longer needs it or has been terminated. When an operating system manages the computer's memory, there are broad tasks to be accomplished

### Device Management

An operating system is a program that *controls the execution of application programs* and acts as an interface between the user of a computer and the computer hardware. A more common definition is that the operating system is the one program running at all times on the computer (usually called the kernel), with all else being application programs. An operating system is concerned with the allocation of resources and services, such as memory, processors, devices, and information. The operating system correspondingly includes programs to manage these resources, such as a traffic controller, a scheduler, memory management module, *I/O programs*, and a file system.

The following functions are used in device management:

- ✓ Sends a control code directly to a specified device driver.
- ✓ Installs a new device. The user is prompted to select the device.
- ✓ Registers the device or type of device for which a window will receive notifications.
- ✓ Closes the specified device notification handle.

### Security and Protection

#### Threats to Protection and Security

A threat is a program that is malicious in nature and leads to harmful effects for the system. Some of the common threats that occur in a system are:

- **Virus**  
Viruses are generally small snippets of code embedded in a system. They are very dangerous and can corrupt files, destroy data, crash systems etc. They can also spread further by replicating themselves as required.
- **Trojan Horse**  
A trojan horse can secretly access the login details of a system. Then a malicious user can use these to enter the system as a harmless being and wreak havoc.
- **Trap Door**  
A trap door is a security breach that may be present in a system without the knowledge of the users. It can be exploited to harm the data or files in a system by malicious people.
- **Worm**  
A worm can destroy a system by using its resources to extreme levels. It can generate multiple copies which claim all the resources and don't allow any other processes to access them. A worm can shut down a whole network in this way.
- **Denial of Service**  
This type of attacks does not allow the legitimate users to access a system. It overwhelms the system with requests so it is overwhelmed and cannot work properly for another user.

## CHAPTER 3: MOBILE PLATFORM

**Mobile Platform** – The overall environment where mobile apps run. It includes the hardware (smartphone/tablet), the mobile OS, and app stores.

- Android – Uses Java/Kotlin, Android Studio, Google Play Store.
- iOS – Uses Swift/Objective-C, Xcode, Apple App Store.

**Mobile Operating System (Mobile OS)** – The software that manages the phone's hardware and provides the interface for apps to work.

**Mobile Application (Mobile App)** – A software program designed to run on a mobile OS to perform specific tasks for users.

- **Native Mobile App** – an application developed *specifically for one mobile operating system* (either Android or iOS) using that system's native programming languages and tools.
  - Android: Developed using Java or Kotlin in Android Studio.
  - iOS: Developed using Swift or Objective-C in Xcode.
  - ✓ Installed through official app stores (Google Play, Apple App Store).
  - ✓ Uses native UI components that match the look and feel of the platform.
  - ✓ Runs directly on the device — does not need a browser.
- **Web Mobile App** – can access through mobile browser (like Chrome, Safari, or Firefox). These apps use the same technologies as websites — HTML, CSS, and JavaScript — but are optimized for mobile viewing.
  - ✓ Runs in a browser (no installation required)
  - ✓ Responsive design adjusts to different screen sizes
  - ✓ Updates automatically when you reload the page
  - ✓ Can be turned into Progressive Web Apps (PWAs), which can work offline and even be pinned on your home screen.
- **Hybrid Mobile App** – It is built using web technologies (HTML, CSS, JavaScript) like a web app, but it's wrapped inside a native container using frameworks such as Apache Cordova or Capacitor. This allows it to be installed on devices and distributed through app stores, just like a native app, while reusing a single codebase for multiple platforms.
  - ✓ One codebase for Android and iOS
  - ✓ Installed through app stores
  - ✓ Can access some native device functions
  - ✓ UI can mimic native design through frameworks (e.g., Ionic components)
- The platform is the whole ecosystem,
- The OS is the backbone that makes the device usable,
- The app is what you install and use on top of it.

**Ionic Stack** – it is a full stack of technologies used for building hybrid mobile applications. **This includes:**

- **Ionic Framework** (UI components)  
It is an open-source UI toolkit for building cross-platform apps using web technologies (HTML, CSS, JavaScript).
  - ✓ Pre-built mobile UI components
  - ✓ Works with Angular, React, or Vue
  - ✓ Can deploy apps to Android, iOS, and web
  - ✓ Ideal for building Hybrid Apps
- **Angular** (frontend frameworks)  
It is a web framework developed by Google used to build dynamic, single-page web applications (SPAs).
  - ✓ Component-based architecture
  - ✓ Uses TypeScript
  - ✓ Supports two-way data binding
  - ✓ Works well with Ionic for hybrid app development
- **Cordova** (bridge to native device features)  
A mobile application development framework that allows developers to use HTML, CSS, and JavaScript to build cross-platform apps.
  - ✓ Wraps web applications into a native container to access device features (camera, GPS, etc.) through plugins.